

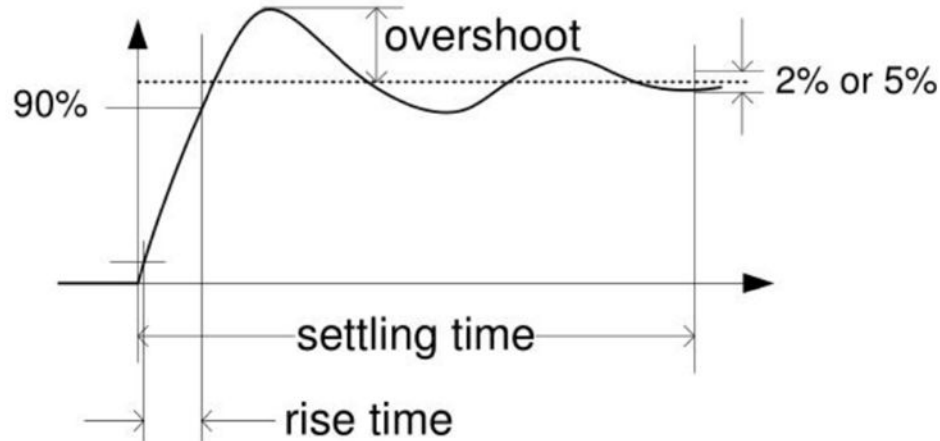
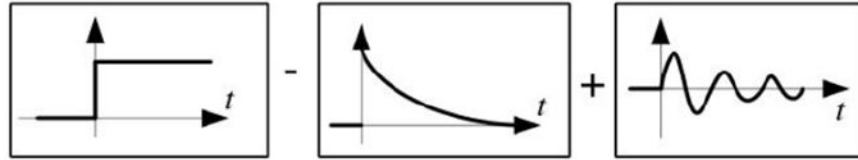
# Introduction to Robotics

## CSE 461

Class Topic: Introduction to Control System Theory (PID)  
Brac University

## 2<sup>nd</sup> order control system

- Typical response to step input is:



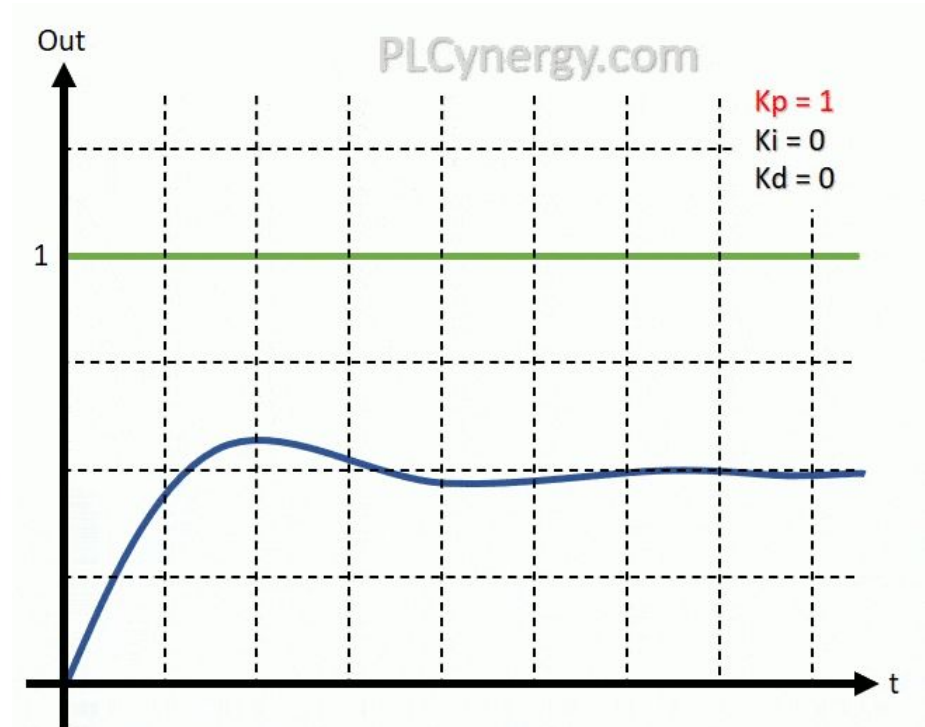
**overshoot** -- % of final value exceeded at first *oscillation*

**rise time** -- time to span from 10% to 90% of the final value

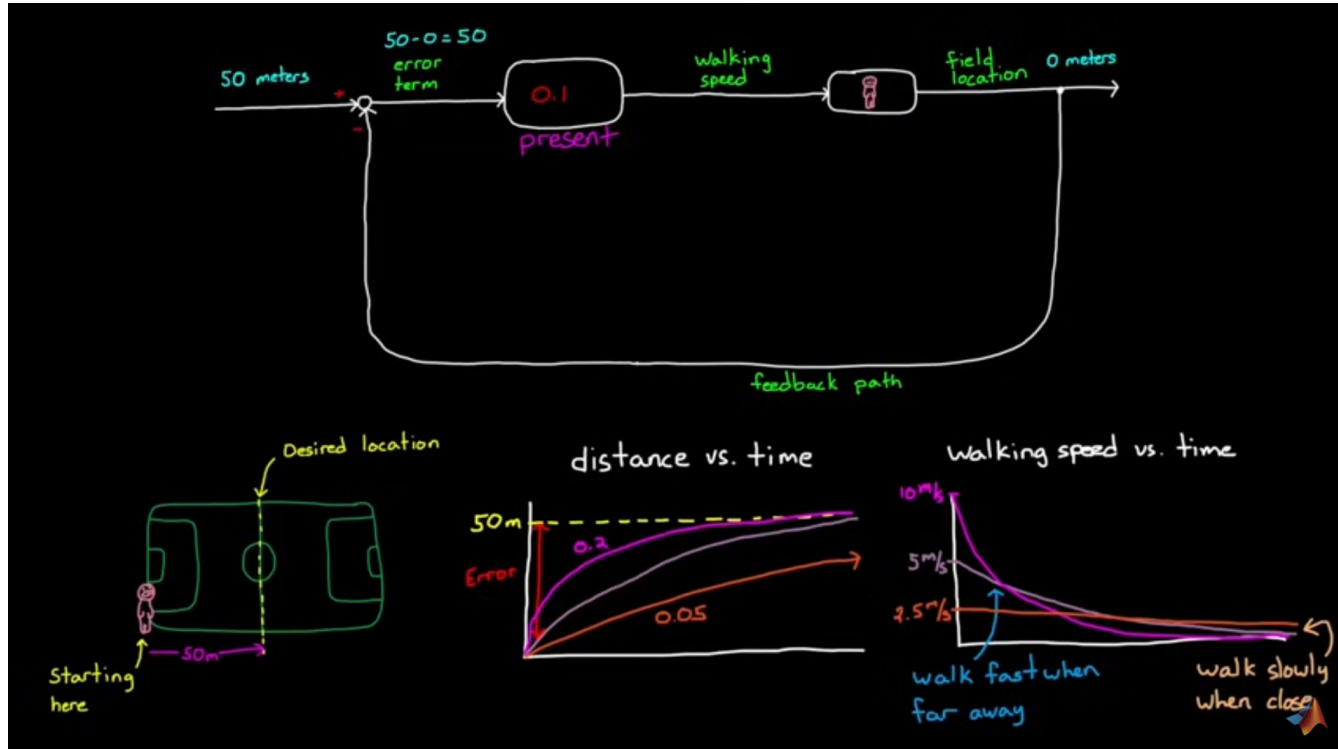
**settling time** -- time to reach within 2% or 5% of the final value

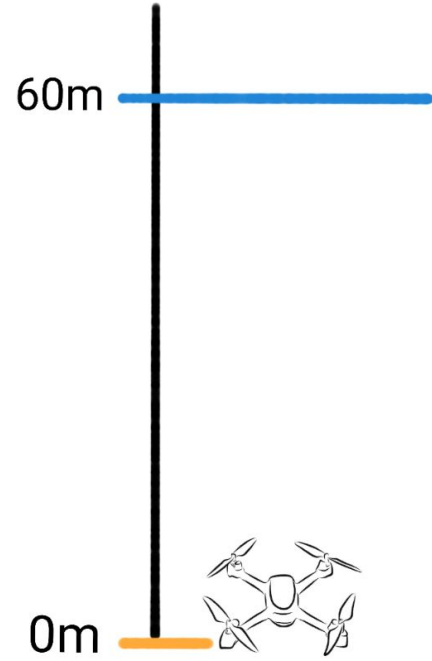
# Effect of PID controller function

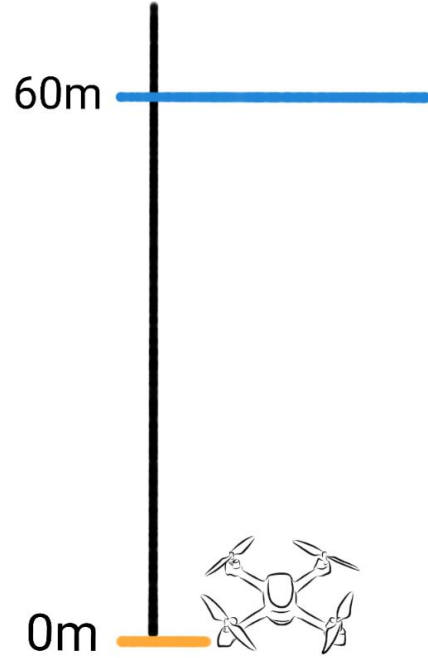
- Proportional Action
  - Simplest Controller Function
- Integral Action
  - Eliminates steady-state error
  - Can cause oscillations
- Derivative Action (“rate control”)
  - Effective in transient periods
  - Provides faster response (higher sensitivity)
  - Never used alone



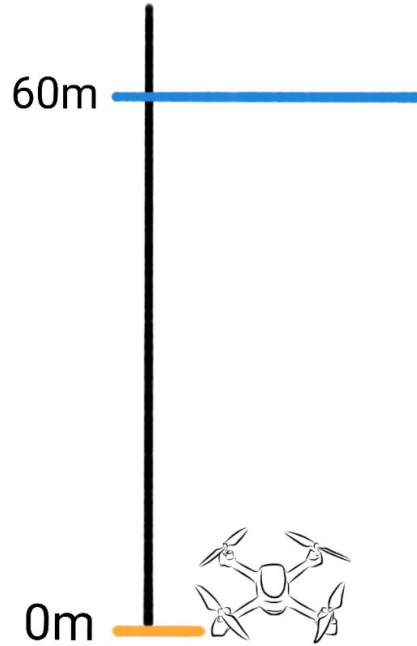
# Proportional works with football field





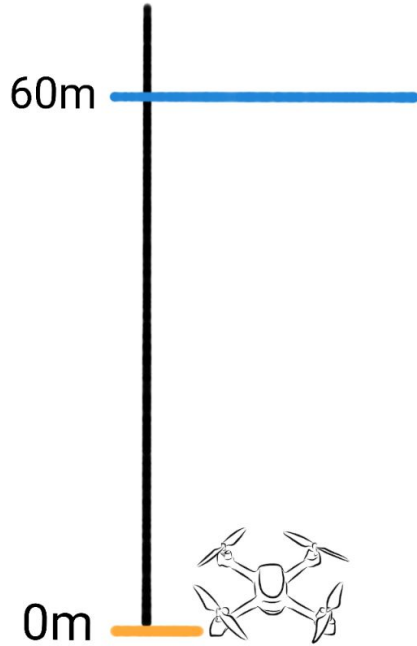


Desired Altitude = 60m  
Actual Altitude = 0m



Desired Altitude = 60m  
Actual Altitude = 0m

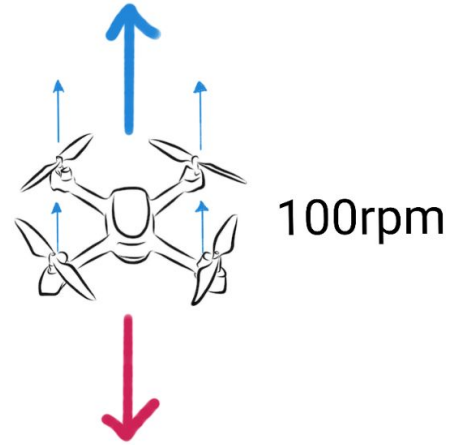
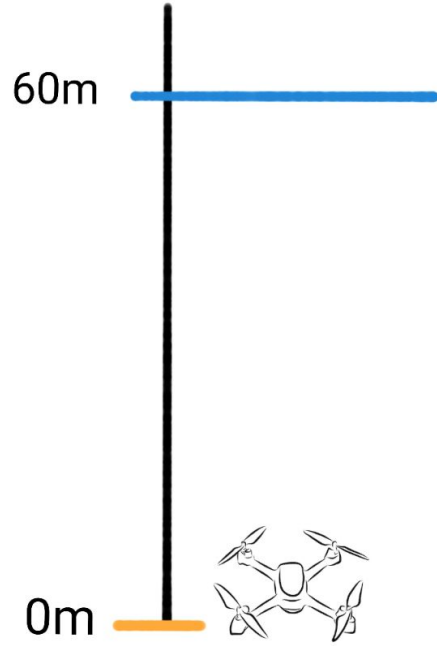
$$\begin{aligned} \text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 0\text{m} \\ &= 60 \end{aligned}$$

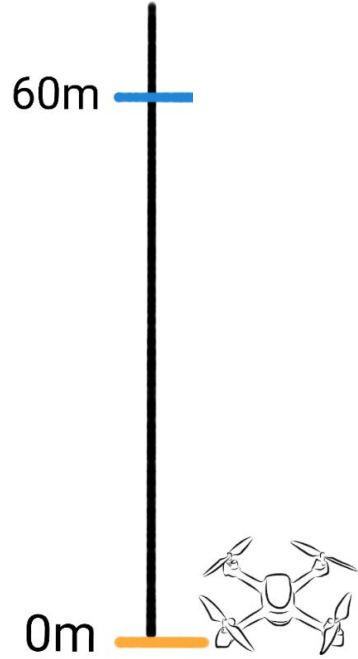
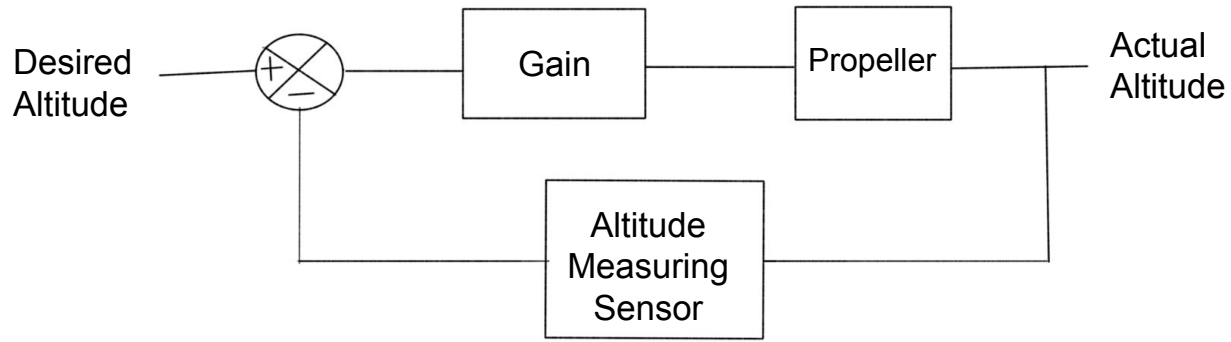


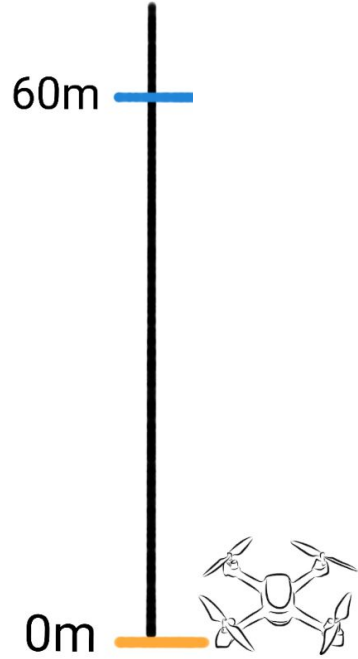
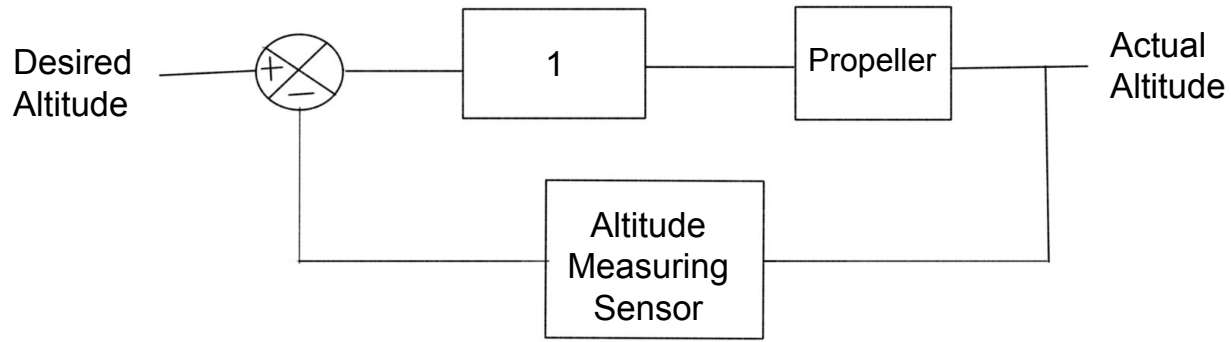
Desired Altitude = 60m  
Actual Altitude = 0m

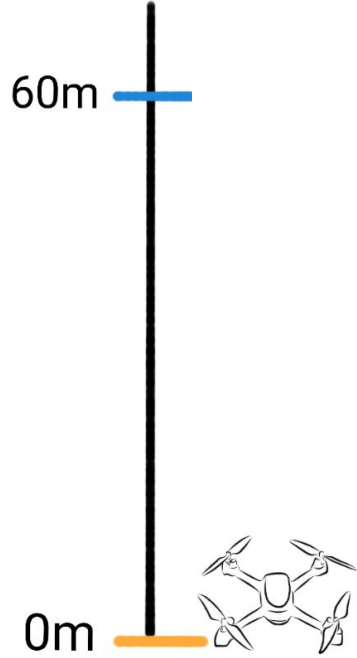
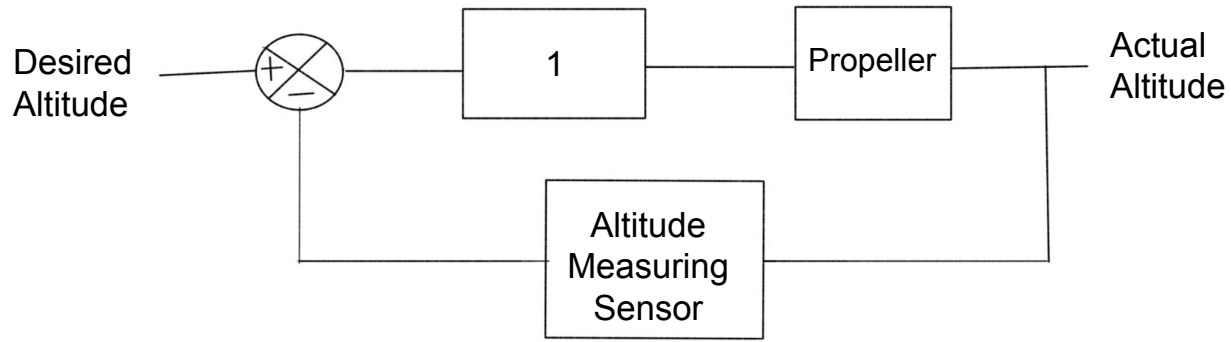
**Desired Altitude - Actual Altitude**  
**= 60m - 0m**  
**= 60**  
**= Error**





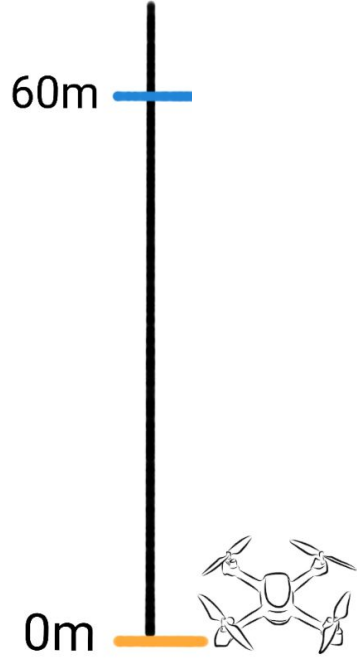
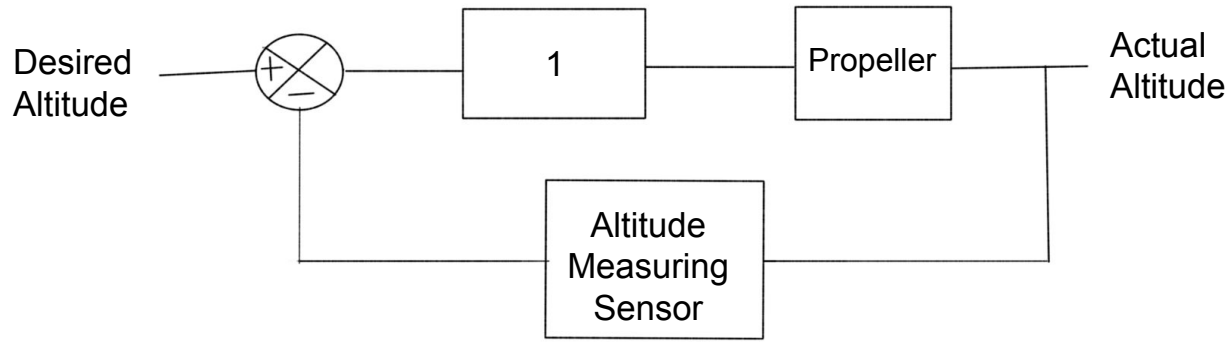






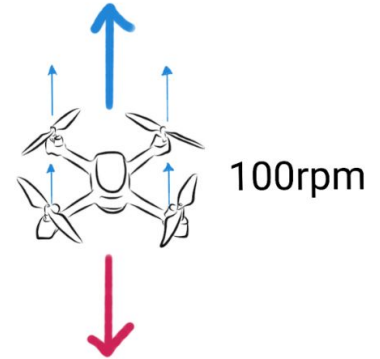
$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 0\text{m} \\ &= 60\end{aligned}$$

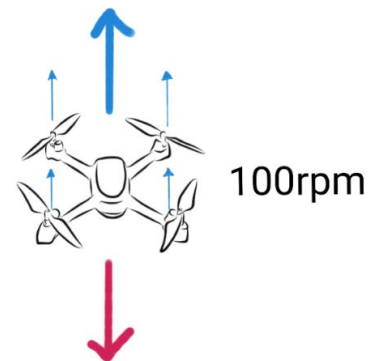
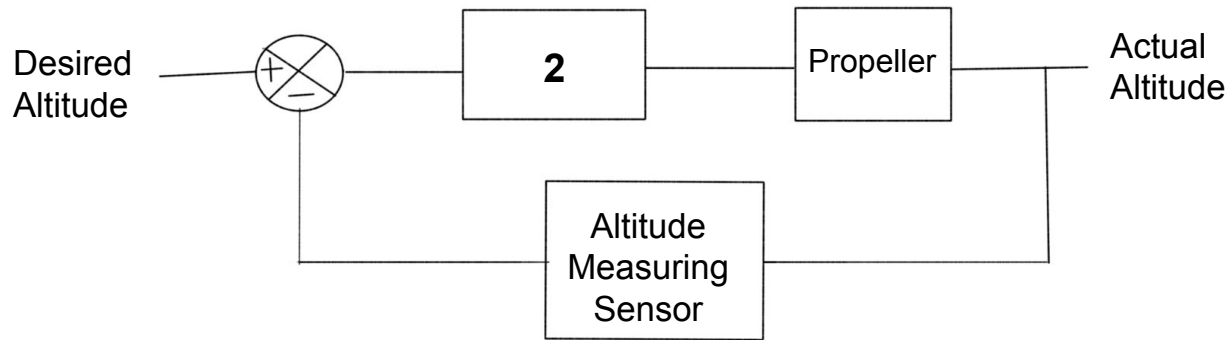
$$\text{Propeller Speed} = 60 * 1 = 60 \text{ rpm}$$

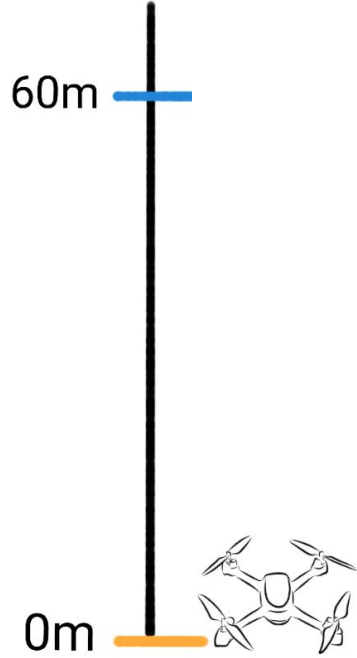
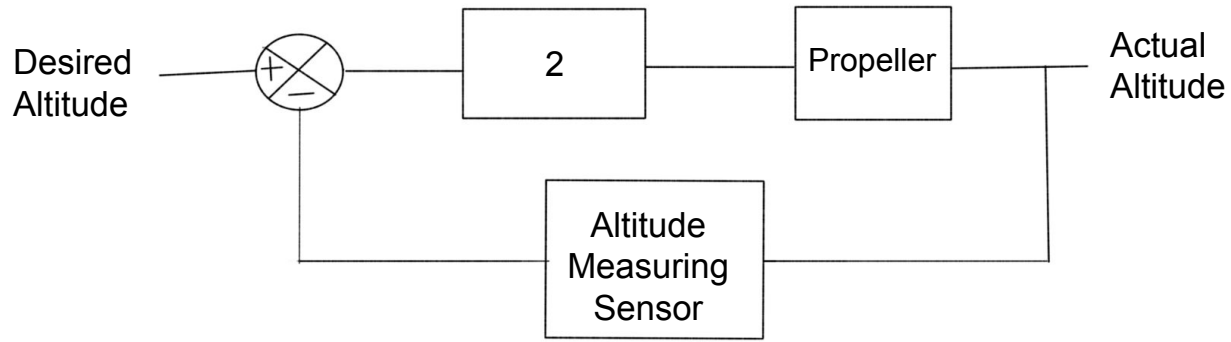


$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 0\text{m} \\ &= 60\end{aligned}$$

$$\text{Propeller Speed} = 60 * 1 = 60 \text{ rpm}$$

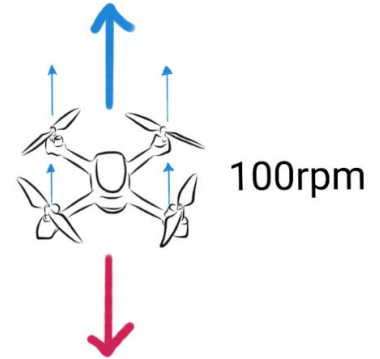


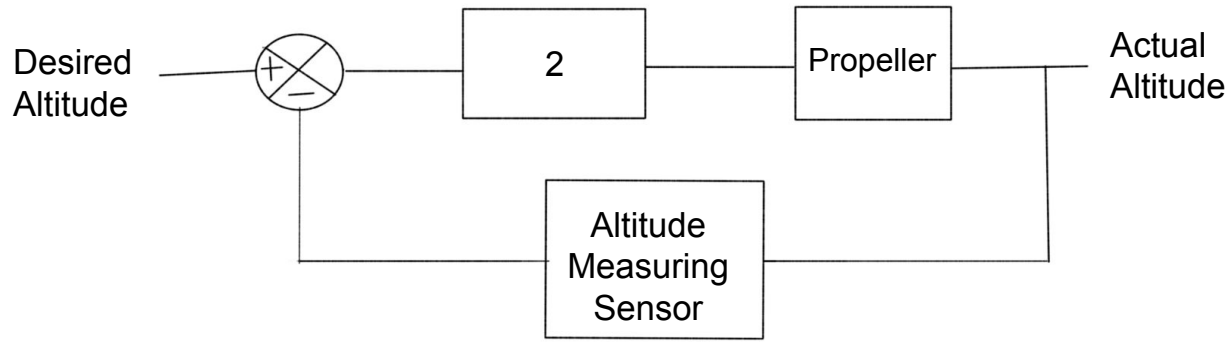




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 0\text{m} \\ &= 60\end{aligned}$$

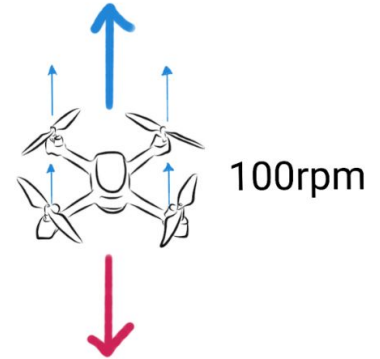
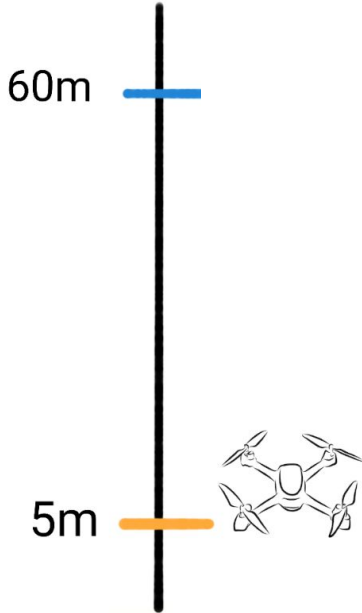
$$\text{Propeller Speed} = 60 * 2 = 120 \text{ rpm}$$



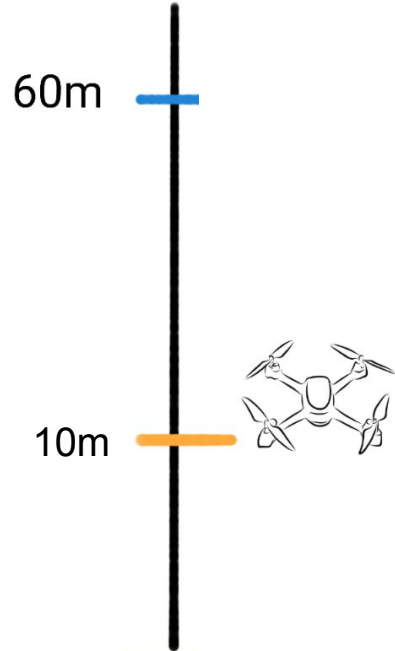
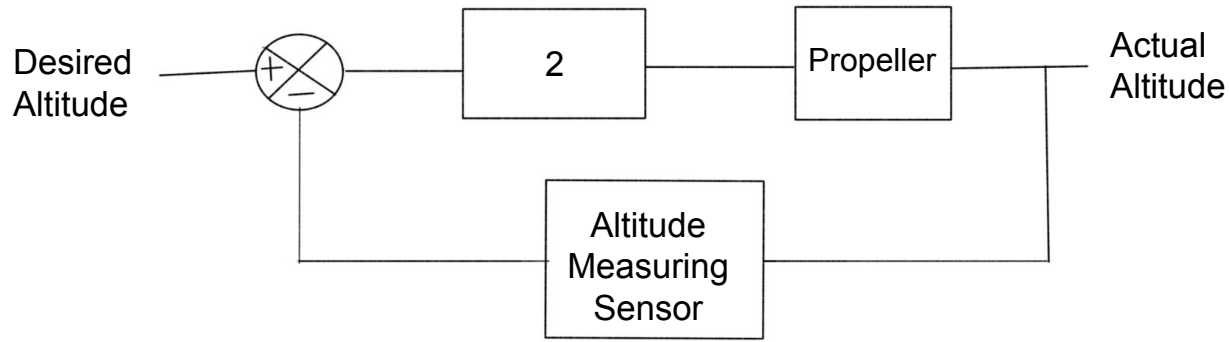


$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 5\text{m} \\ &= 55\end{aligned}$$

$$\text{Propeller Speed} = 55 * 2 = 110 \text{ rpm}$$

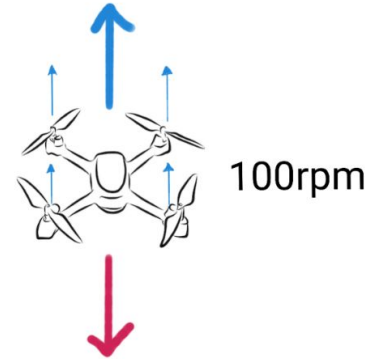


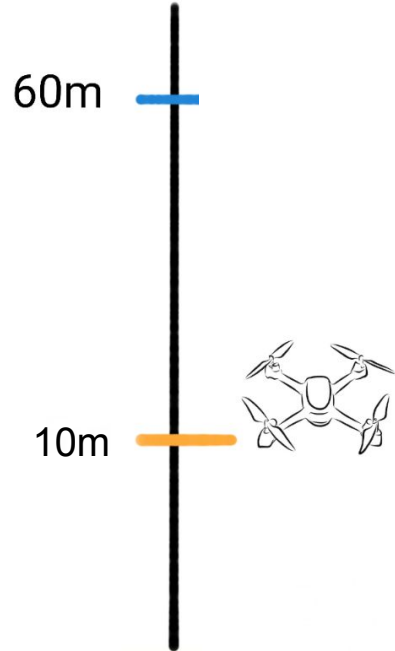
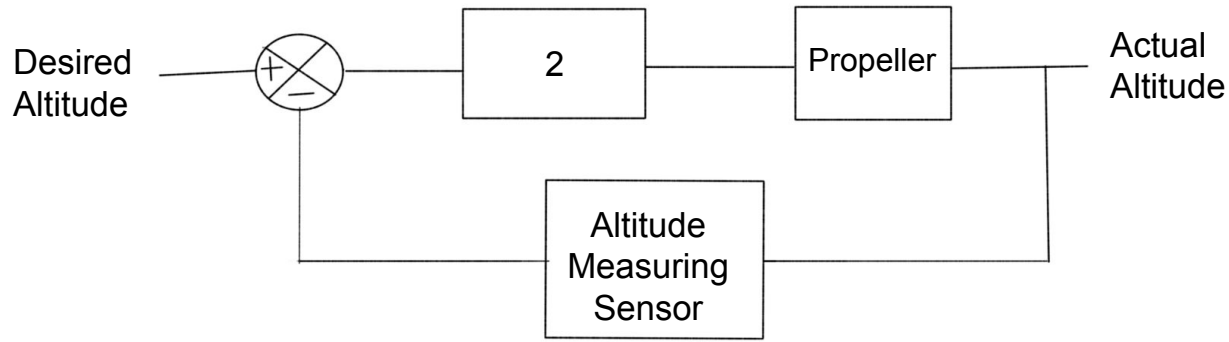




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 10\text{m} \\ &= 50\end{aligned}$$

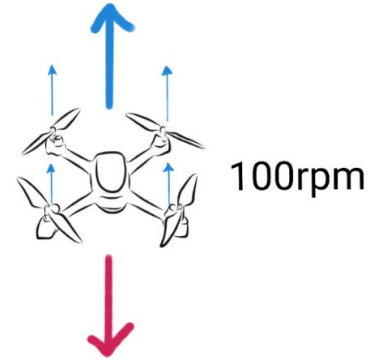
$$\text{Propeller Speed} = 50 * 2 = 100 \text{ rpm}$$

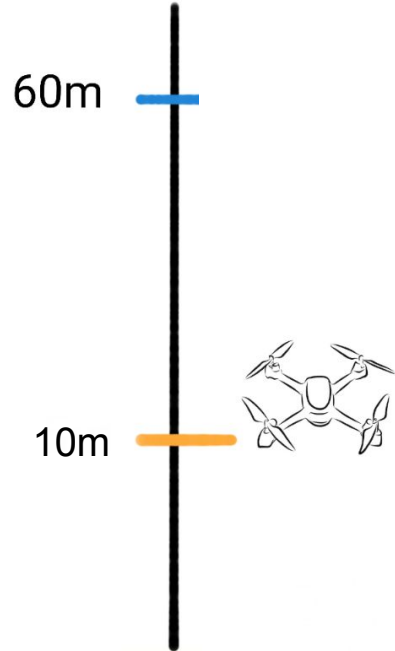
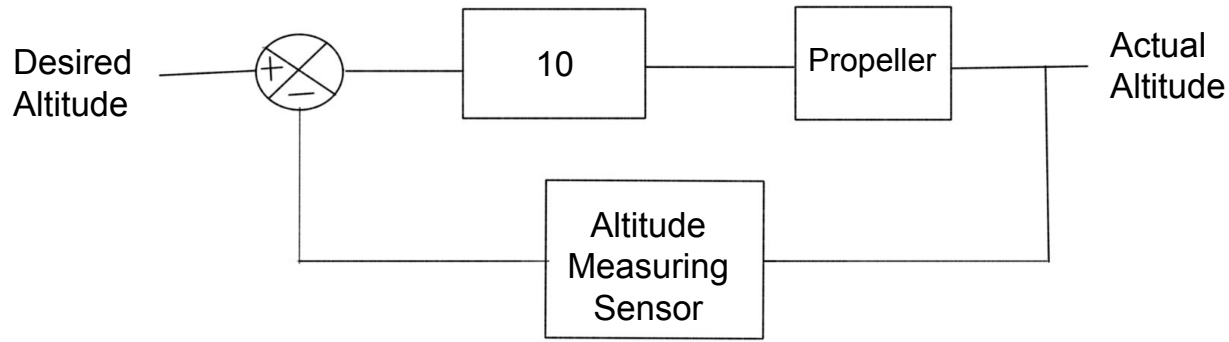




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 10.1111\text{m} \\ &= 49.9999\end{aligned}$$

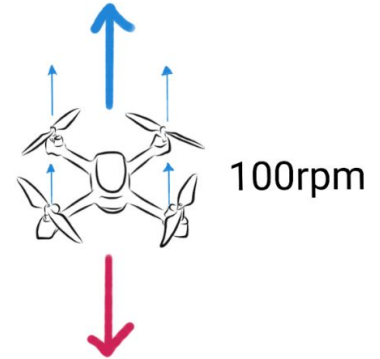
$$\begin{aligned}\text{Propeller Speed} &= 49.99 \times 2 = 99.99 \\ &\text{rpm}\end{aligned}$$

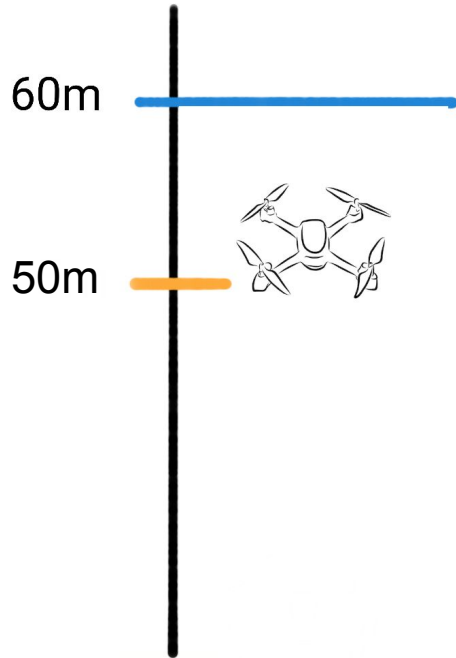
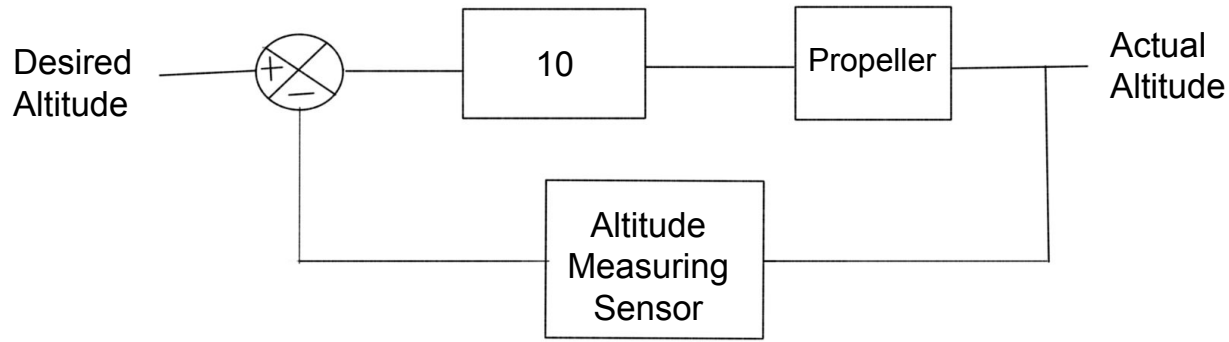




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 10\text{m} \\ &= 50\end{aligned}$$

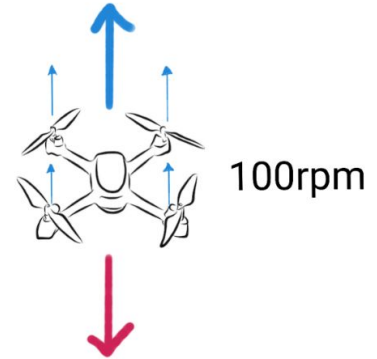
$$\text{Propeller Speed} = 50 * 10 = 500 \text{ rpm}$$

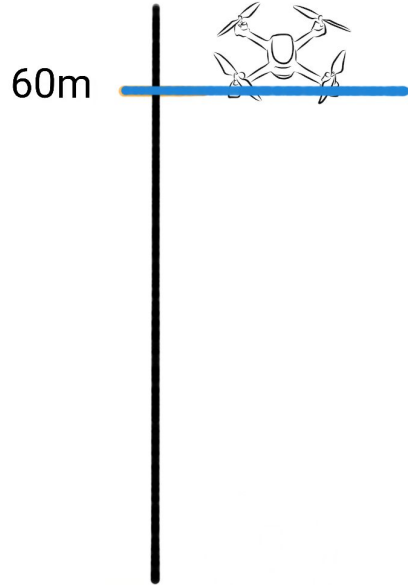
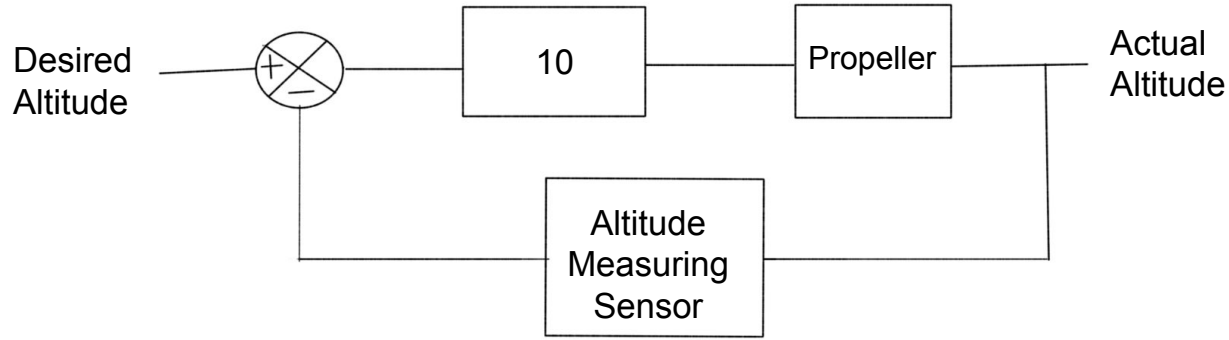




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 50\text{m} \\ &= 10\end{aligned}$$

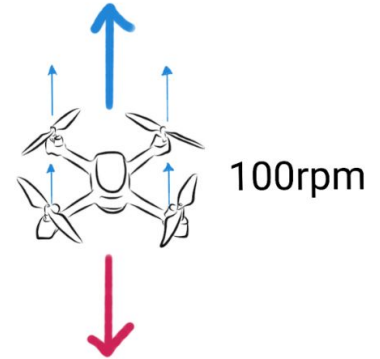
$$\text{Propeller Speed} = 10 * 10 = 100 \text{ rpm}$$

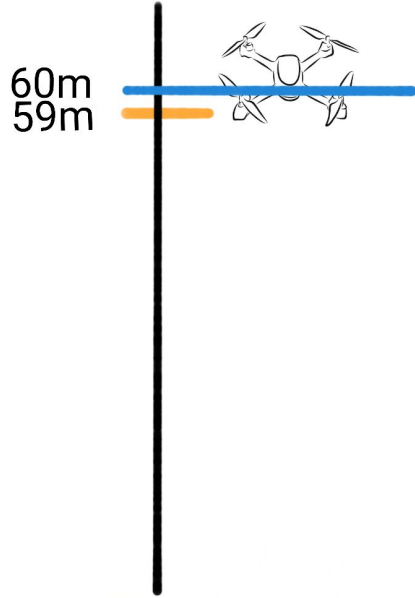
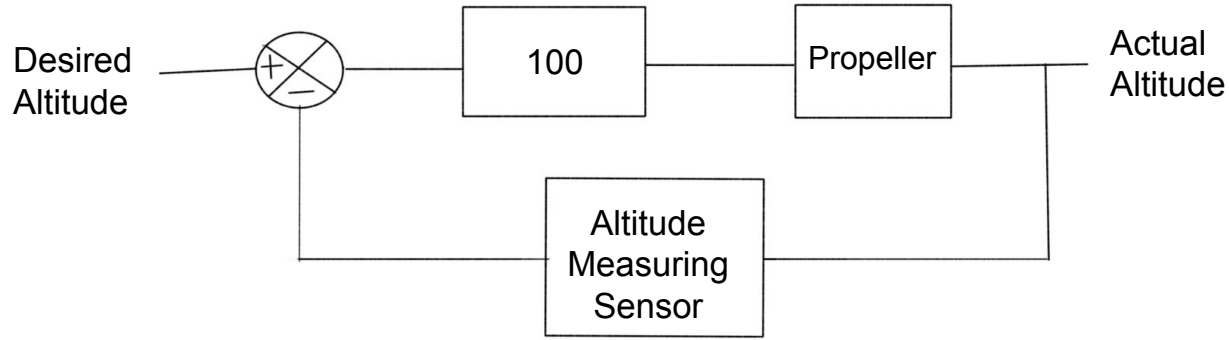




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 60\text{m} \\ &= 0\end{aligned}$$

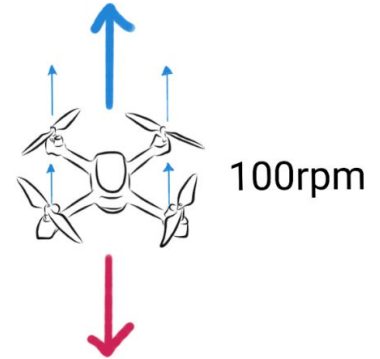
$$\text{Propeller Speed} = 0 * 10 = 0 \text{ rpm}$$

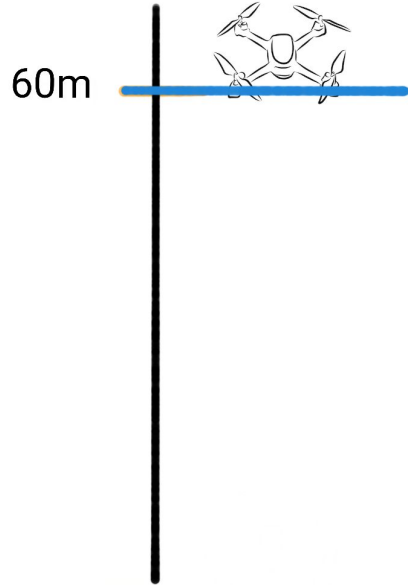
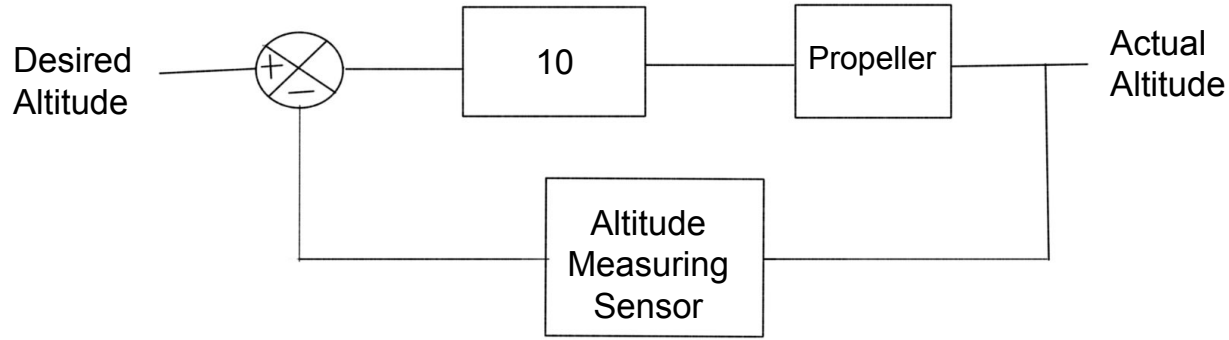




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 59\text{m} \\ &= 1\end{aligned}$$

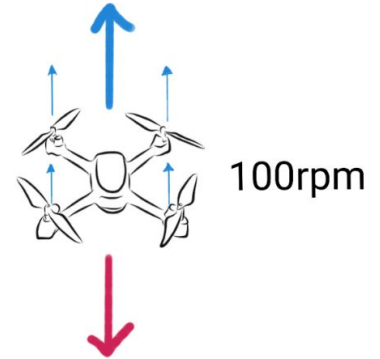
$$\text{Propeller Speed} = 1 * 100 = 100 \text{ rpm}$$

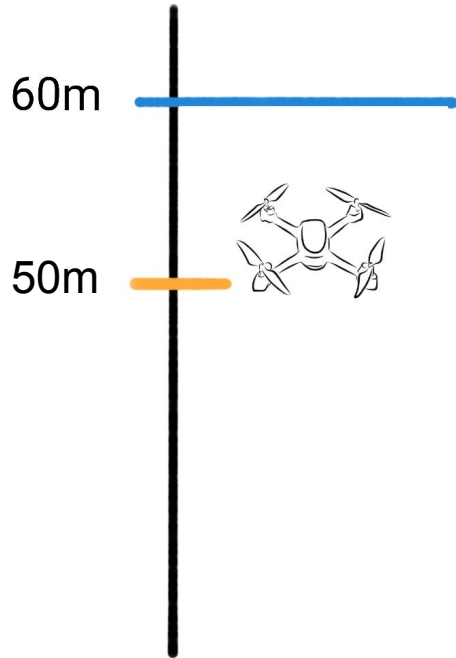
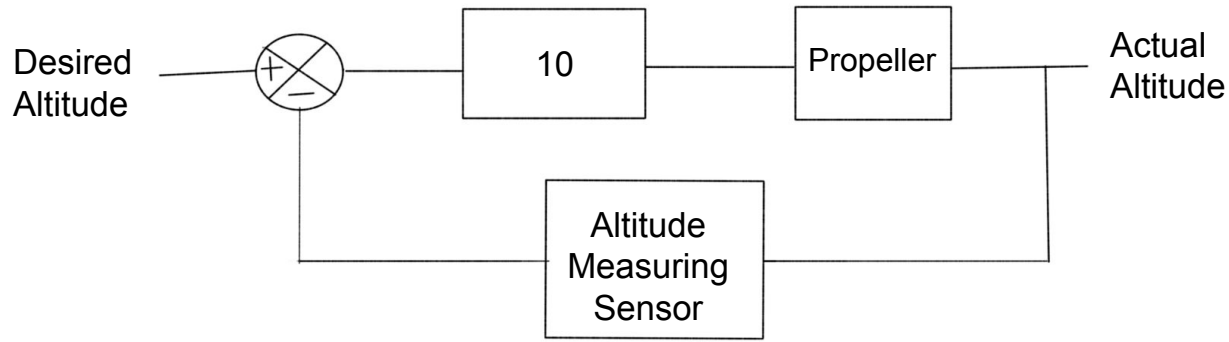




$$\begin{aligned}\text{Desired Altitude} - \text{Actual Altitude} \\ &= 60\text{m} - 59.1\text{m} \\ &= .9\end{aligned}$$

$$\text{Propeller Speed} = .9 * 10 = 90 \text{ rpm}$$

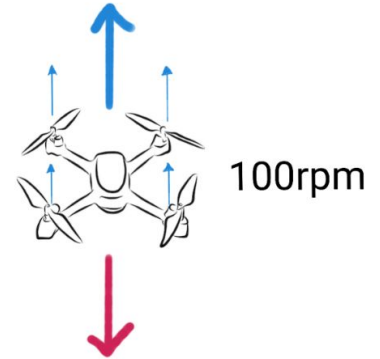




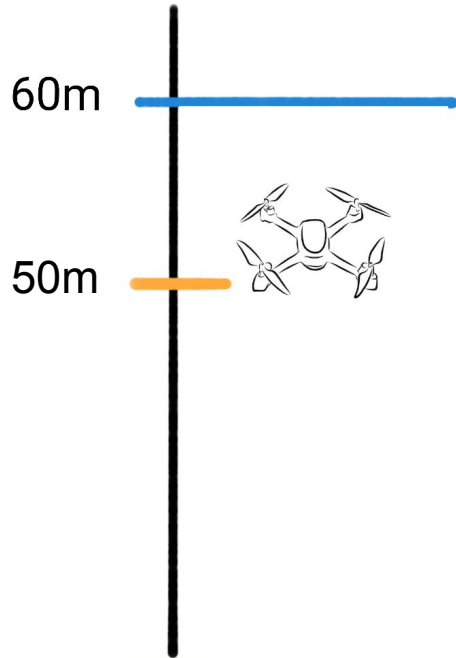
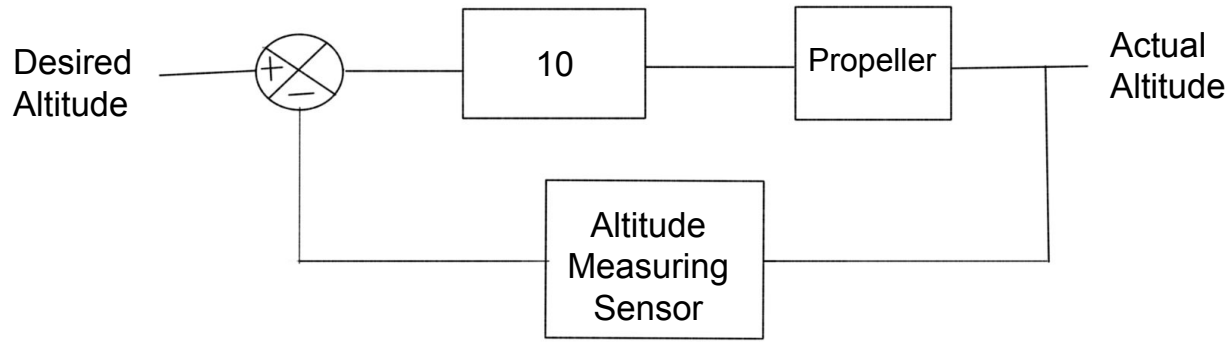
$$\text{Propeller Speed} = 10 * 10 = 100 \text{ rpm}$$

$$\text{Error} = 10$$

$$\text{Output} = K_p * \text{error}$$





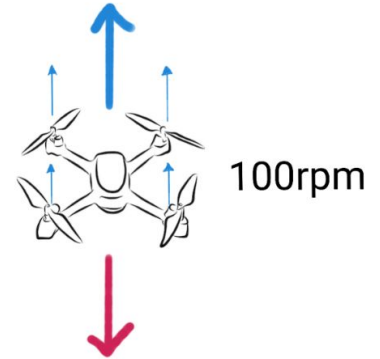


$$\text{Propeller Speed} = 10 * 10 = 100 \text{ rpm}$$

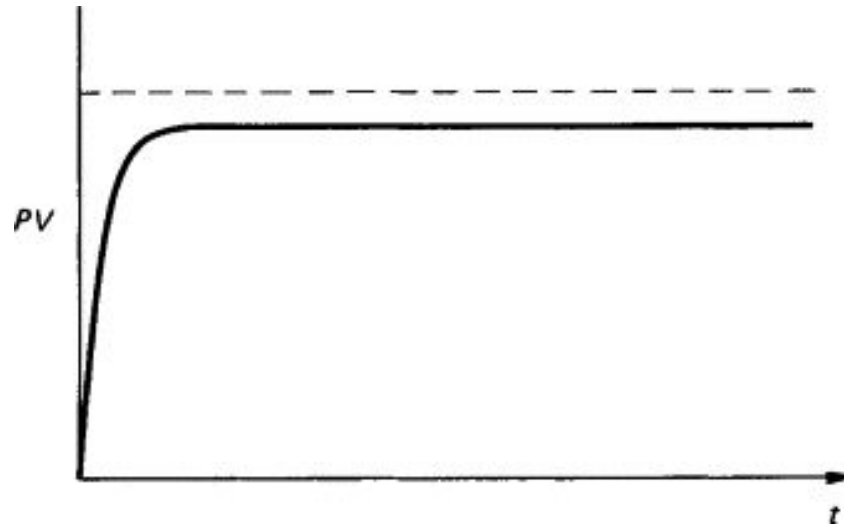
$$\text{Error} = 10$$

$$\text{Output} = K_p * \text{error}$$

**Proportional Control**



## Proportional Control



**Steady-State error**

# PID Controller

Proportion al control :  $u(t) = K_p e(t)$

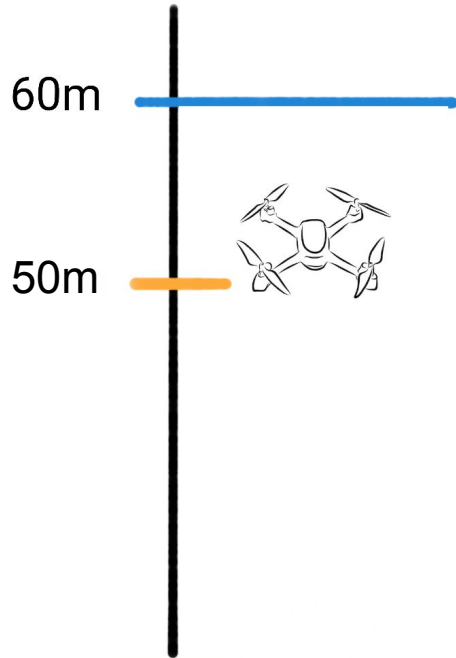
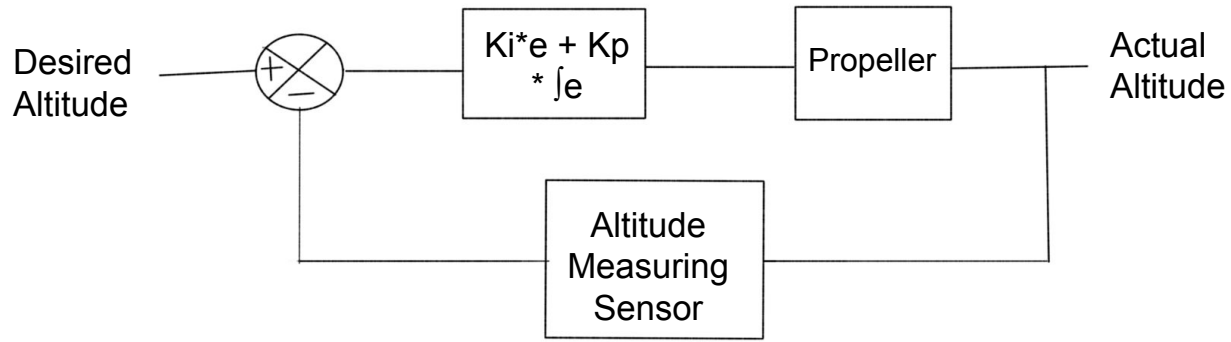
Integral control :  $u(t) = K_i \int_0^t e(t) dt$

Differenti al control :  $u(t) = K_d \frac{d}{dt} e(t)$

- It produces an output, which is the combination of the outputs of proportional , integral & derivative controllers

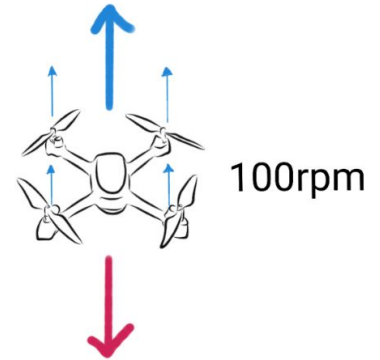
$$u(t) \propto e(t) + \int e(t) + \frac{d}{dt} e(t)$$

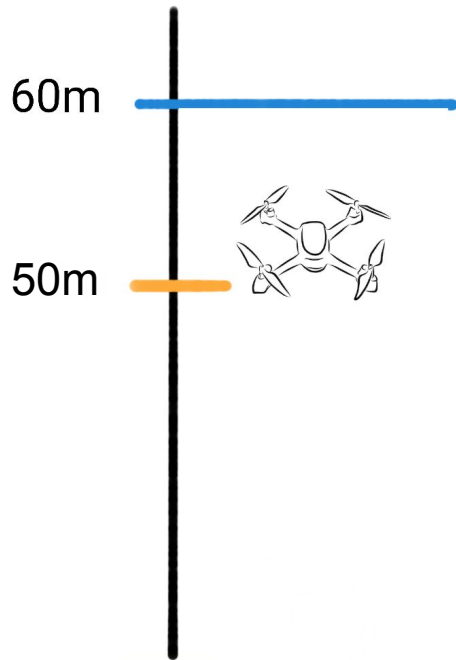
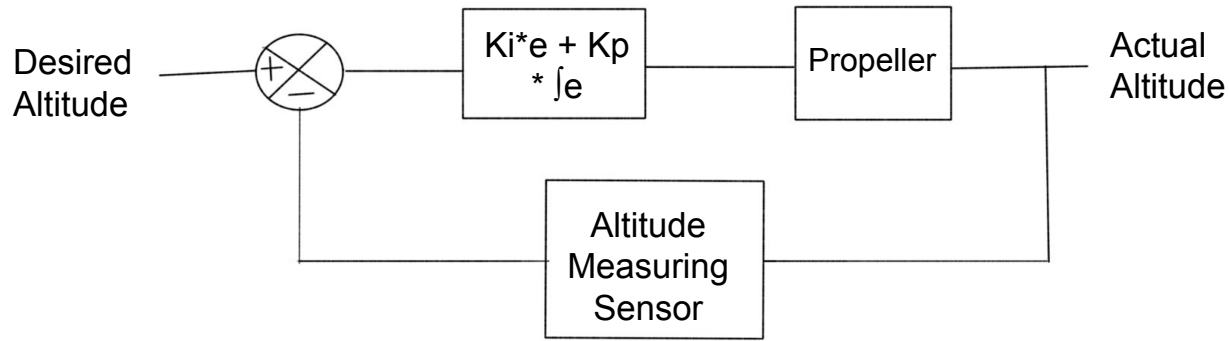
$$\gg u(t) = K_P e(t) + K_I \int e(t) + K_D \frac{d}{dt} e(t)$$



$$\text{Propeller Speed} = 10 * 10 = 100 \text{ rpm}$$

$$\text{Error} = 10$$



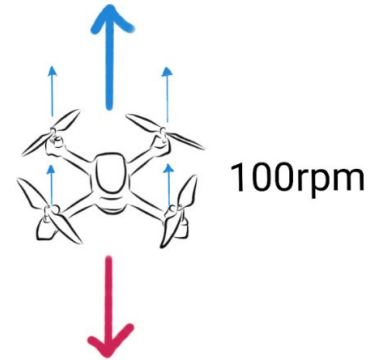


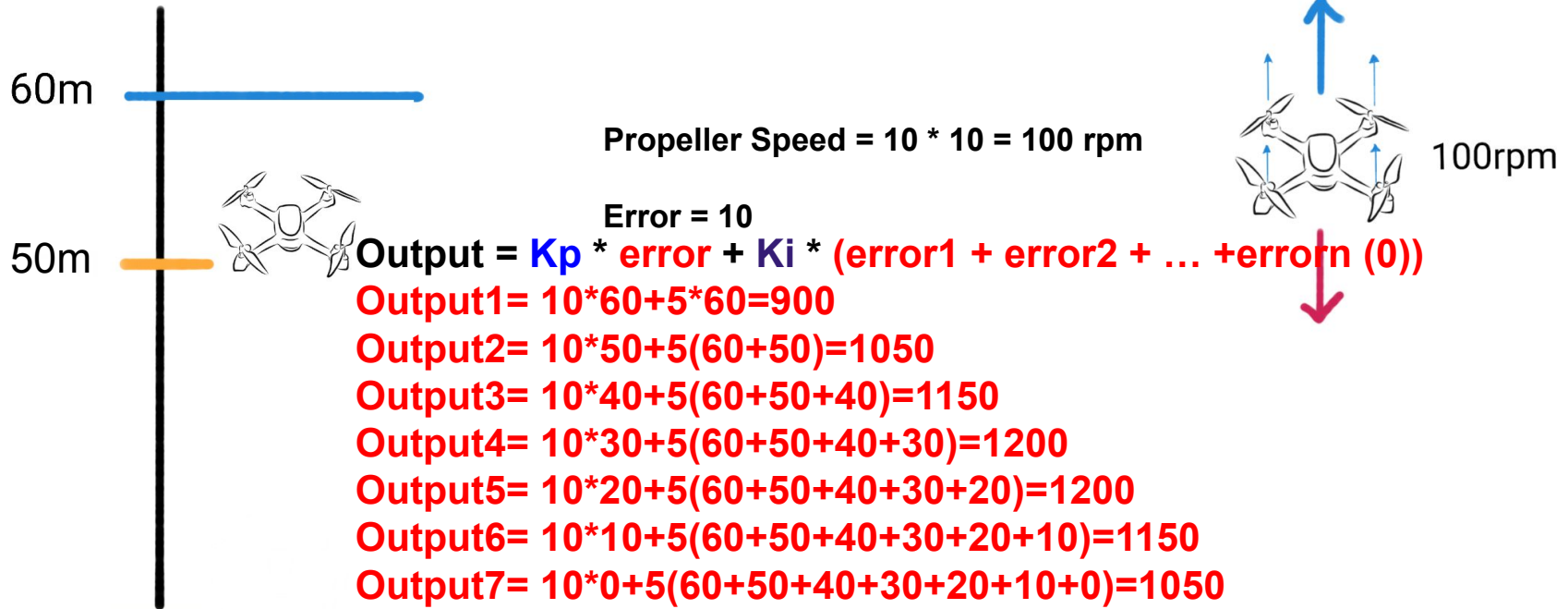
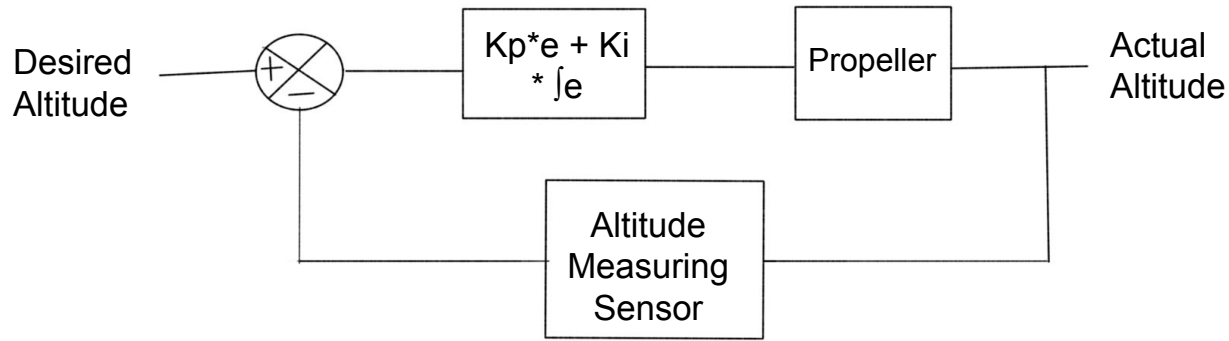
$$\text{Propeller Speed} = 10 * 10 = 100 \text{ rpm}$$

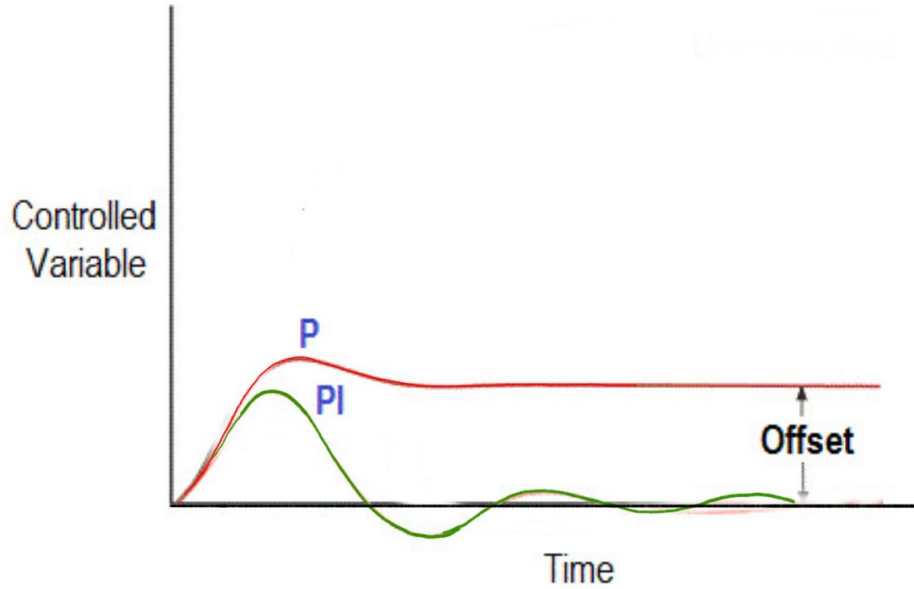
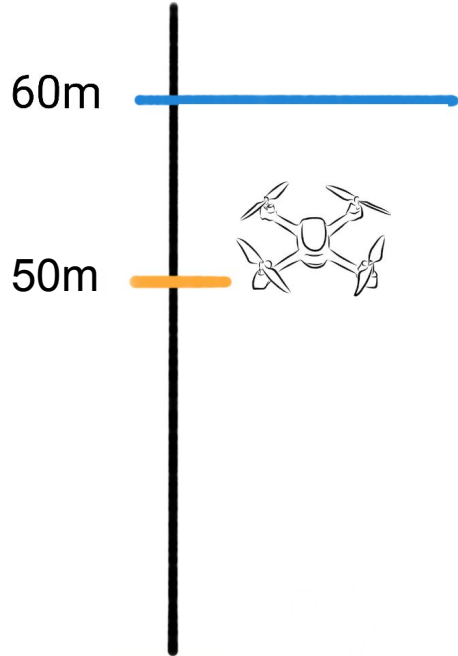
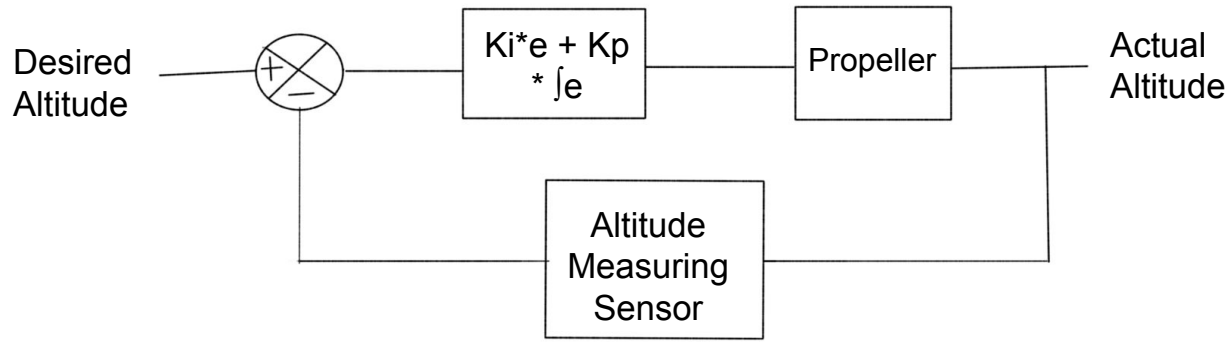
$$\text{Error} = 10$$

$$\text{Output} = K_p * \text{error} + K_i * (\text{error}_1 + \text{error}_2 + \dots + \text{error}_N(0))$$

**Proportional - Integral Control**

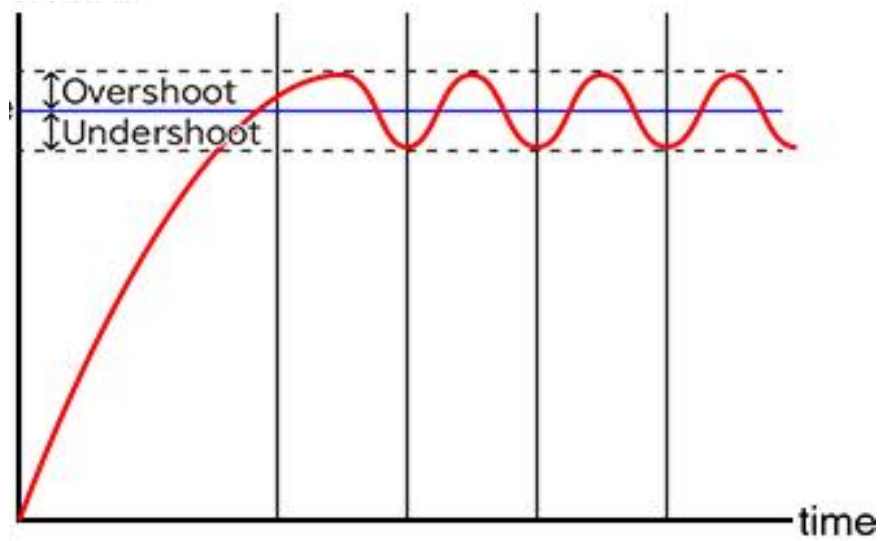


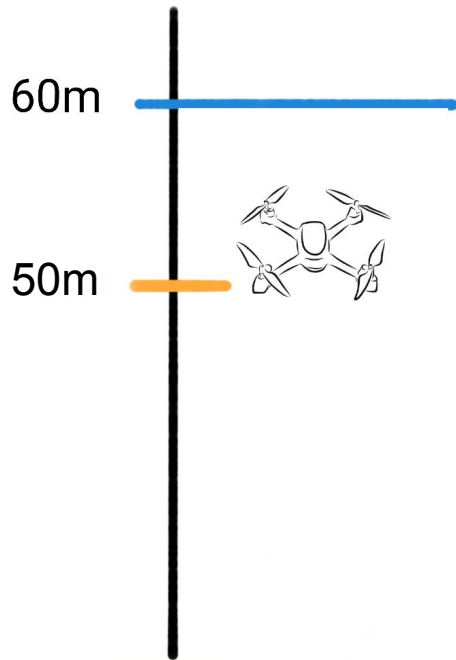
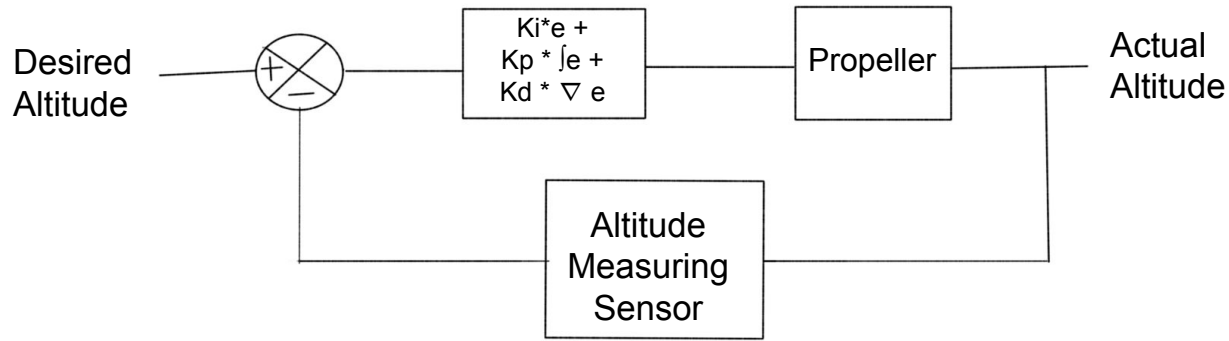






## Proportional Control

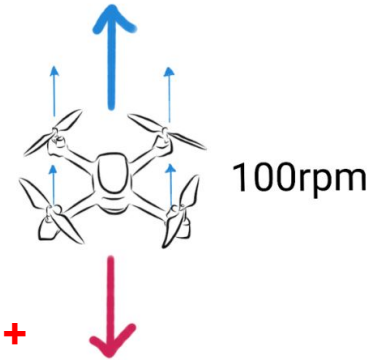


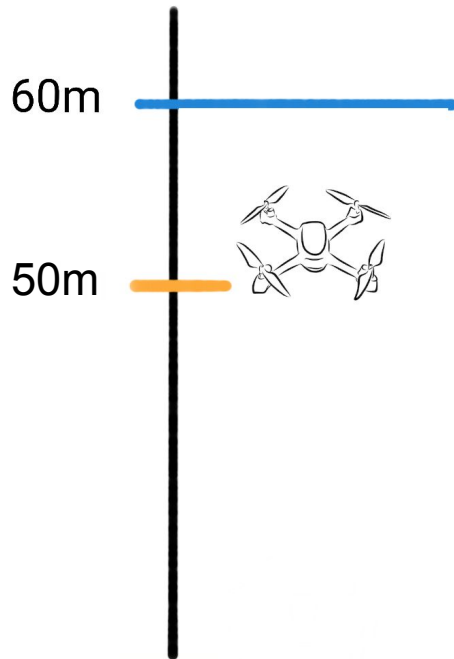
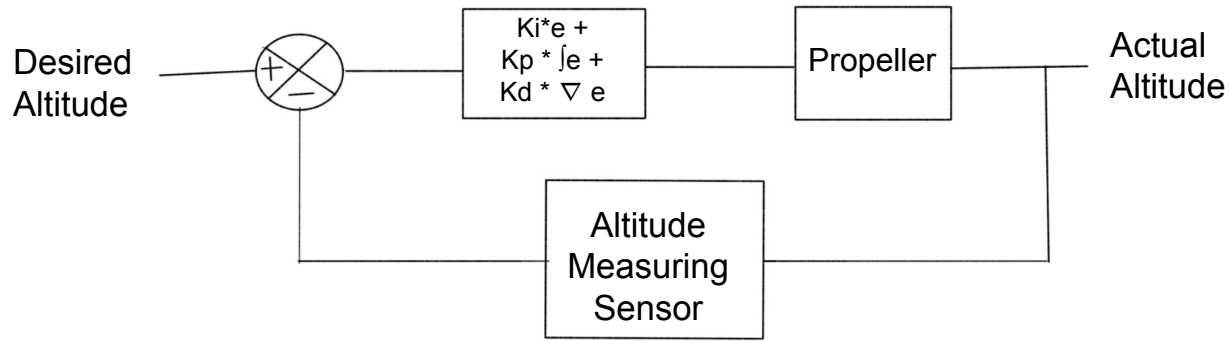


$$\text{Propeller Speed} = 10 * 10 = 100 \text{ rpm}$$

$$\text{Error} = 10$$

$$\text{Output} = K_p * \text{error} + K_i * (\text{error1} + \text{error2} + \text{error3}) + K_d * (\text{error3} - \text{error2})$$

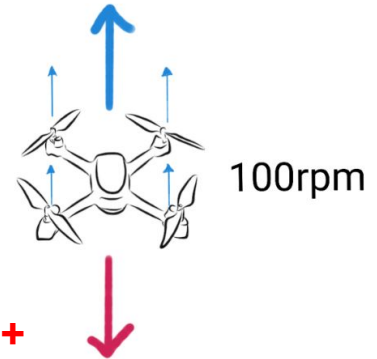




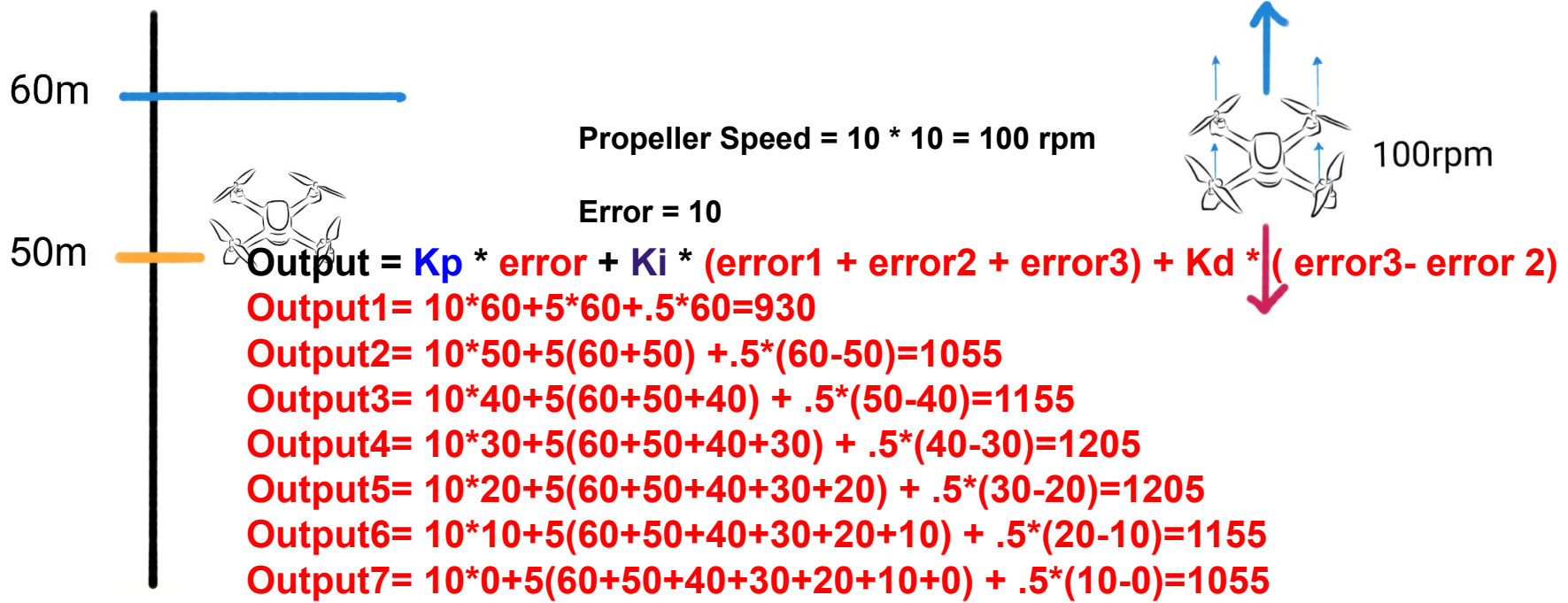
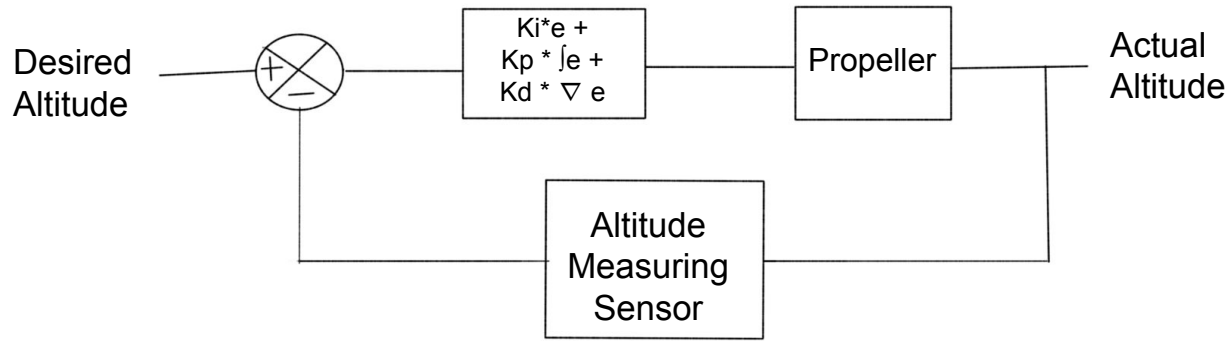
Propeller Speed =  $10 * 10 = 100 \text{ rpm}$

Error = 10

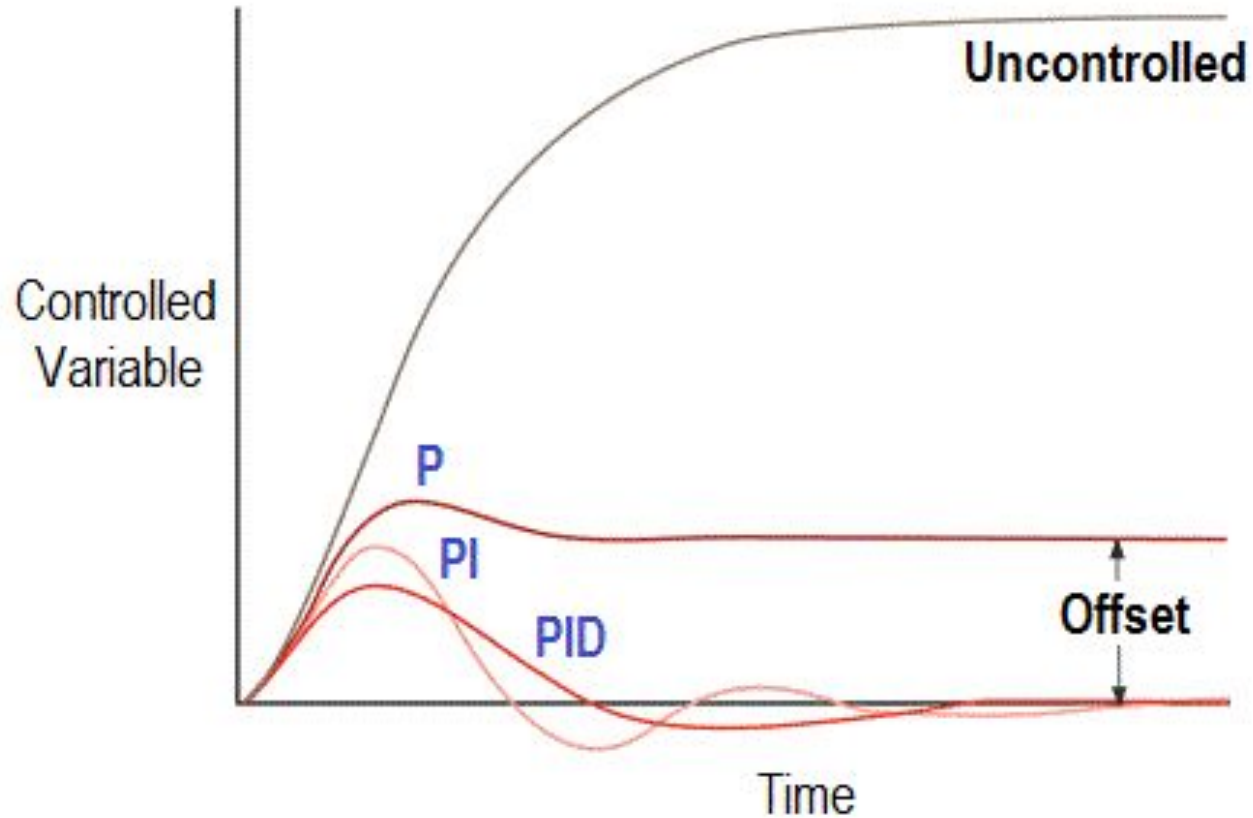
Output =  $K_p * \text{error} + K_i * (\text{error1} + \text{error2} + \text{error3}) + K_d * (\text{error3} - \text{error2})$



Proportional - Derivative - Integral Control  
(PID) Control



## Proportional - Derivative - Integral Control (PID) Control



# Effect of Controller Functions

- Proportional Action

Simplest Controller Function, The P term helps to reduce the steady-state error and improve the system's responsiveness. However, too high of a proportional gain can lead to instability and oscillations in the system's response.

- Integral Action

Eliminates steady-state error, The I term helps to improve the system's stability and eliminate any bias in the system. However, too high of an integral gain can lead to overshoot and instability in the system's response.

- Derivative Action (“rate control”)

Effective in transient periods, The D term helps to improve the system's stability and reduce the effects of disturbances in the system. However, too high of a derivative gain can lead to noise amplification and instability in the system's response.

## How to get the PID parameter values ?

- If we know the transfer function, analytical methods can be used (e.g., root-locus method) to meet the transient and steady-state specs.
- When the system dynamics are not precisely known, we must resort to experimental approaches.

## Ziegler-Nichols Rules for Tuning PID Controller

Using only Proportional control, turn up the gain until the system oscillates without dying down, i.e., is marginally stable. Assume that  $K$  and  $P$  are the resulting gain and oscillation period, respectively.

Then Use

for P control

$$K_p = 0.5 K$$

for PI control

$$K_p = 0.45 K$$

$$K_i = 1.2 / P$$

for PID control

$$K_p = 0.6 K$$

$$K_i = 2.0 / P$$

$$K_d = P / 8.0$$

Ziegler-Nichols Tuning  
for second or higher  
order systems

# Generating PID parameters

- **Auto-Tuning Algorithms:**
  - Relay-based Auto-Tuning
  - A relay is introduced in the control loop.
  - Observes the system's oscillatory response.
  - Calculates PID parameters based on oscillation characteristics.
- **Trial and Error:** Iteratively adjust parameters based on system response.



# Advantages and Applications of PID control

## Advantages:

- Simple: easy to understand.
- Effective: Accurate and stable, even in dynamic environments.
- Robust: Can be adopted to different robot systems

## Application:

- Arm positioning
- Robot Navigation
- Speed Control
- Balance control

# Next Class

Machine Learning